

Documento Técnico — Predicción de Precio de Viviendas

Universidad Nacional de Costa Rica — Sede Regional Brunca, Campus Pérez Zeledón
Curso: Inteligencia Artificial · Proyecto Final

Estudiantes:

- Esteban Granados Sibaja
- Juan Carlos Camacho Solano
- Allan Vargas Torrer
- Francisco Mora Cabezas

Tipo de problema: Regresión (aprendizaje supervisado) **Dataset:** Ames Housing (Kaggle — *House Prices: Advanced Regression Techniques*)

Introducción

Este documento describe el diseño, implementación y evaluación de un sistema de predicción de precios de viviendas basado en técnicas de Machine Learning. El sistema modela el precio de venta de una propiedad (SalePrice) a partir de 22 características físicas, de calidad y ubicación, utilizando el dataset Ames Housing.

Se implementaron y compararon dos modelos: **Regresión Lineal** (línea base interpretable) y **Random Forest** (modelo de conjunto no lineal). El proyecto cubre el ciclo completo: formulación del problema, análisis exploratorio, preprocesamiento con pipeline reproducible, entrenamiento, evaluación experimental con métricas estándar de regresión, y un sistema interactivo con interfaz Streamlit y predicción por consola.

1. Formulación del problema

1.1 Definición

El sistema estima el **precio de venta de una vivienda** (SalePrice) a partir de sus características físicas, de calidad y de ubicación. Como el objetivo es una cantidad monetaria **continua**, el problema es de **regresión** y no de clasificación.

1.2 Variables

- **Variable objetivo (y):** SalePrice — precio final de venta (continuo).
- **Variables predictoras (X):** 22 variables seleccionadas:
 - **16 numéricas:** GrLivArea, TotalBsmtSF, 1stFlrSF, 2ndFlrSF, LotArea, YearBuilt, YearRemodAdd, OverallQual, OverallCond, GarageCars, GarageArea, FullBath, HalfBath, BedroomAbvGr, TotRmsAbvGrd, Fireplaces.
 - **6 categóricas:** MSZoning, Neighborhood, KitchenQual, BsmtQual, ExterQual, Foundation.

1.3 Justificación técnica del enfoque

- El objetivo es numérico y continuo → métricas de error continuo (MAE, MSE, RMSE, R^2).
- Existe una relación funcional entre los atributos de la vivienda y su precio.
- Se comparan **dos modelos complementarios**: Regresión Lineal (línea base interpretable) y Random Forest (no lineal, capta interacciones).
- **Meta operativa**: minimizar el RMSE en datos no vistos manteniendo un R^2 alto y estable bajo validación cruzada.

1.4 Encuadre como agente inteligente (PEAS)

Componente	En este proyecto
Performance	RMSE/MAE bajos y R^2 alto en datos no vistos
Environment	Mercado inmobiliario de Ames, Iowa (2006–2010)
Actuators	Precio estimado + comparación entre modelos
Sensors	Vector de 22 características ingresadas por el usuario

El entorno es **parcialmente observable** (no se conocen todos los factores del mercado), **estocástico**, **estático** respecto a una consulta individual y **continuo** en sus variables y salida.

2. Dataset y análisis exploratorio (EDA)

El dataset original tiene **1460 filas y 81 columnas**. Se redujo a 22 variables con criterio técnico (señal predictiva, baja redundancia, interpretabilidad). Hallazgos principales del EDA (detalle y gráficos en el notebook):

Hallazgo	Evidencia	Decisión derivada
SalePrice con fuerte asimetría positiva	skew = 1.88 (original) → 0.12 con log1p; media 180.921 > mediana 163.000	Tratar outliers; aplicar log1p al objetivo (implementado en producción vía TransformedTargetRegressor, ver §5.1.2)
Predictores más fuertes	Mayor correlación: OverallQual, GrLivArea, GarageCars/Area, TotalBsmtSF	Confirma la selección de variables
Multicolinealidad	GarageCars–GarageArea, TotRmsAbvGrd–GrLivArea	Escalado para la Regresión Lineal; descartar redundantes
Outliers	31 viviendas sobre el límite IQR de GrLivArea; casas grandes con precio bajo	Winsorización (clip 1%/99%) en lugar de eliminar filas
Sesgo geográfico	El vecindario más caro vale 3.58x el más barato	Neighborhood es predictor clave y fuente de sesgo
No linealidad	Saltos de precio crecientes según OverallQual	Justifica usar Random Forest

2.1 Tratamiento de outliers (justificación)

Se decidió **winsorizar** (recortar al percentil 1/99 con QuantileClipper) en lugar de eliminar registros porque: (1) la muestra es pequeña (1460 filas) y eliminar descarta información real; (2) el clipping acota el impacto de los extremos sin perder la observación; (3) los límites se ajustan **solo con el conjunto de entrenamiento** dentro del pipeline, evitando fuga de información.

3. Preprocesamiento

Todo el preprocesamiento vive dentro de un **Pipeline de scikit-learn con ColumnTransformer**, lo que garantiza reproducibilidad y reutilización idéntica en entrenamiento, validación cruzada, inferencia CLI y aplicación Streamlit (sin fuga de información).

Flujo: DomainImputer → RareCategoryGrouper → QuantileClipper → ColumnTransformer.

Etapa	Qué hace	Justificación
DomainImputer	Reglas de dominio (sin sótano/garaje ⇒ valores 0 y etiquetas explícitas) + flags has_garage/has_bsmt	Los nulos son estructurales , no aleatorios
Imputación numérica	SimpleImputer(median)	Robusta frente a asimetría
Imputación categórica	SimpleImputer(constant="Missing")	Conserva la señal de "ausencia"
RareCategoryGrouper	Agrupar categorías con frecuencia < 1% en "Other"	Evita columnas dummy inestables
QuantileClipper	Winsorización 1%/99%	Control de outliers (ver 2.1)
OneHotEncoder(handle_unknown="ignore")	Codifica categóricas	Tolera categorías no vistas en inferencia
StandardScaler	Escala numéricas (solo en Regresión Lineal)	Necesario para el modelo lineal; innecesario para árboles

División de datos: 80% entrenamiento / 20% prueba (random_state=42). La validación cruzada (KFold=5) se ejecuta **solo sobre el conjunto de entrenamiento**.

4. Modelos implementados

- **Modelo 1 — Regresión Lineal:** línea base interpretable y de baja varianza.

- **Modelo 2 — Random Forest:** modelo no lineal, capta interacciones sin ingeniería manual extensa.

Ajuste de hiperparámetros: el **notebook** documenta la búsqueda con GridSearchCV (KFold=5, scoring por RMSE). Grillas: fit_intercept/positive (lineal); n_estimators, max_depth, min_samples_split, min_samples_leaf, max_features (Random Forest).

Modelo final seleccionado: Random Forest. Random Forest fue seleccionado como modelo final del sistema porque obtuvo el mejor desempeño predictivo en el conjunto de prueba. La Regresión Lineal con transformación logarítmica (log1p) se reconoce como una alternativa metodológicamente sólida, más estable y con menor riesgo de sobreajuste, pero no fue seleccionada como modelo operativo debido a que presentó un desempeño predictivo inferior en test. La diferencia entre modelos no es estadísticamente significativa ($p \approx 0.13$), por lo que se despliega el mejor estimador puntual en test (Random Forest) y se muestran ambos al usuario para una decisión informada.

Modelo servido (fuente de verdad). El artefacto de producción (models/*.joblib) y todas las métricas oficiales provienen de src/train.py, que entrena la configuración **base** del Random Forest (n_estimators=300, random_state=42). El tuning del notebook es **reproducible desde producción** con python -m src.train --tune, que ejecuta el mismo GridSearchCV y persiste el modelo ajustado. Las métricas de esta sección corresponden al modelo base servido.

5. Evaluación experimental

5.1 Métricas en el conjunto de prueba (modelo base servido)

Random Forest — métricas de reports/metrics_comparison.csv (fuente de producción, consumida también por la app). **Regresión Lineal** — configuración **base (sin log1p)**, usada como línea base de comparación. *Nota metodológica:* la LR de producción usa el preprocesador completo con log1p (configs D/E del estudio de ablación, RMSE ≈ 28.581 , $R^2 \approx 0.894$); el experimento con preprocesador simplificado + log1p (config B, RMSE ≈ 27.339) se documenta en §5.1.2. La diferencia entre configs B y D/E refleja que el OrdinalEncoder y AgeFeatureCreator no mejoran la Regresión Lineal en este dataset.

Modelo	MAE	RMSE test	R ² test
Random Forest	17.212	27.308	0.9028
Regresión Lineal (base, sin log1p)	18.171	32.001	0.8665

En la **partición de prueba** el Random Forest obtiene menor RMSE. El MAPE es **10.7%** (RF) y **10.2%** (Lineal). **Sin embargo, esta tabla por sí sola no justifica elegir un modelo** (ver 5.1.1): el test es una sola partición.

5.1.1 Selección por validación cruzada e incertidumbre (anti sesgo de selección)

Elegir el modelo por su RMSE en el test introduce **sesgo de selección** (se usa el mismo conjunto para elegir y para reportar). La selección se hace por **validación cruzada KFold=5 sobre el train**, reportando la incertidumbre:

Modelo	CV RMSE (media $\pm$ std)	Test RMSE
Regresión Lineal	29.994 $\pm$ 4.396	32.001

Random Forest	30.007 ± 5.023	27.308
---------------	----------------	--------

Hallazgo clave: por validación cruzada la Regresión Lineal y el Random Forest son **prácticamente equivalentes** (sus intervalos media ± std se solapan ampliamente). En una validación cruzada **repetida (5×5 = 25 estimaciones)** la diferencia RF vs Lineal **no es estadísticamente significativa** (prueba *t* pareada, **p ≈ 0.13**; RF gana en 18/25 particiones). La ventaja del Random Forest en el test es, en buena parte, **un efecto de esa partición concreta**, no una superioridad robusta. Conclusión honesta: **el Random Forest es el mejor estimador puntual, pero no supera de forma significativa a la Regresión Lineal.**

5.1.2 Transformación del objetivo (log1p): la mejora con mayor evidencia

El precio tiene fuerte asimetría positiva (skew 1.88) y residuos heterocedásticos (5.2). El tratamiento estándar —y con el que se evalúa este dataset en Kaggle— es modelar $\log_{1p}(\text{SalePrice})$ y revertir con `expm1`. Implementado vía `TransformedTargetRegressor` y disponible con `python -m src.train --log-target`:

Nota: "LR + log" corresponde al **experimento de ablación config B** (preprocesador básico + $\log_{1p}$ en objetivo, RMSE 27.339). El modelo LR de **producción** usa el preprocesador completo (configs D/E: + `OrdinalEncoder` + `AgeFeatureCreator`) y obtiene RMSE 28.581 — el efecto de esas etapas no mejora la LR en este dataset (ver §5.1). "RF (raw)" corresponde al modelo de producción (config G del estudio de ablación), verificado en `reports/metrics_comparison.csv`.

Modelo	Test RMSE	R ² test	Gap train→test
Regresión Lineal + log (config B)	27.339	0.9026	1.8%
Random Forest + log	27.674	0.9002	59.7%
Regresión Lineal (raw, config A)	32.001	0.8665	12.1%
Random Forest (raw, config G — producción)	27.308	0.9028	59.7%

Evidencia (CV repetida 5×5): la transformación log mejora a la Regresión Lineal de forma **estadísticamente significativa frente a sí misma sin transformar** (LR+log vs LR raw: mejora media ≈ 2.061, **p ≈ 0.002**) y **reduce su sobreajuste de 12.1% a 1.8%**. El Random Forest **no** mejora con log (p ≈ 0.94: es invariante a la escala). Sin embargo, las diferencias de RMSE de **test** entre las mejores configuraciones (RF config G ≈ 27.308 y LR+log config B ≈ 27.339) son del orden de **0.1%, dentro del ruido de muestreo y no estadísticamente significativas** (la comparación principal RF vs Lineal da p ≈ 0.13); por tanto **no constituyen una base para preferir LR+log como modelo operativo.**

Decisión de modelo final. El aporte de $\log_{1p}$ es, sobre todo, **mayor estabilidad y menor sobreajuste** del modelo lineal, no una superioridad predictiva robusta en test. Por ello **el modelo final del sistema es Random Forest** (mejor desempeño puntual en test); **Regresión Lineal + log se documenta como una alternativa metodológicamente sólida —más estable y mejor calibrada (gap 1.8%)— pero no fue seleccionada como modelo operativo.** Es reproducible con `python -m src.train --log-target`.

5.2 Análisis de residuos y errores

- **Residuos:** media cercana a 0 (predicciones aproximadamente insesgadas), pero con **heterocedasticidad** (la dispersión crece en precios altos). El histograma de residuos es simétrico con colas pesadas por viviendas atípicas.

- **Reales vs predichos:** buena alineación con la diagonal; tendencia a **subestimar viviendas de lujo** (pocos ejemplos).
- **Errores grandes:** se concentran en propiedades caras/atípicas → el sistema es más confiable en el rango de precio medio.

5.3 Diagnóstico de sobreajuste (RMSE train vs test)

Modelo	RMSE train	RMSE test	Gap
Regresión Lineal	28.137	32.001	3.864 (12.1%)
Random Forest	11.012	27.308	16.296 (59.7%)

El Random Forest presenta un gap del ~60% (memoriza parte del ruido del entrenamiento), frente al ~12% del modelo lineal. Este gap es la **principal limitación reconocida del modelo final**: aunque su RMSE de test es el mejor, generaliza con menos margen del que aparenta, por lo que la confianza en el sistema se respalda con la validación cruzada (5.1.1) y no solo con el train. La **transformación log reduce el gap del modelo lineal a 1.8%** (5.1.2): por eso Regresión Lineal + log se documenta como la **alternativa mejor calibrada**, aunque **no fue la seleccionada como modelo operativo** (su ventaja en test no es estadísticamente significativa).

6. Sistema interactivo

- **Notebook:** widget ipywidgets para simular escenarios y comparar predicciones.
- **Aplicación Streamlit (app/):** modo rápido (3 variables + defaults) y modo completo (22 variables), con validación de entradas, manejo de errores, simulador A/B de mejoras, consenso ponderado entre modelos, contexto por percentiles e historial de corridas.

7. Análisis crítico

1. **Limitaciones del dataset:** una sola ciudad (Ames, Iowa) y un período histórico (2006–2010); los patrones no se trasladan automáticamente a otras regiones ni a condiciones actuales.
2. **Sobreajuste:** Random Forest tiende a sobreajustar con muestras pequeñas; se monitorea con gap train-test y validación cruzada.
3. **Sesgos:** geográfico (concentración en ciertos vecindarios) y temporal.
4. **Escalabilidad:** ampliar variables y grillas crece de forma no lineal en costo computacional; convendría búsquedas más eficientes (p. ej. aleatoria).
5. **Consideraciones éticas:** la predicción no es verdad absoluta de mercado; usar Neighborhood de forma determinista puede reforzar **desigualdades territoriales**. El sistema debe presentarse como apoyo a la decisión, no como valoración oficial, y con cautela explícita en propiedades de lujo o atípicas.

8. Conclusiones

- Se construyó una solución de regresión **completa, reproducible y modular**: formulación → EDA → preprocesamiento con Pipeline → 2 modelos → validación cruzada → tuning → evaluación con análisis de residuos → sistema interactivo → análisis crítico.
- **Modelo final del sistema: Random Forest.** Fue seleccionado como modelo final porque obtuvo el **mejor desempeño predictivo en el conjunto de prueba** (RMSE $\approx$ 27.308, $R^2 \approx$

0.903). La **Regresión Lineal con transformación logarítmica (log1p)** se reconoce como una **alternativa metodológicamente sólida, más estable y con menor riesgo de sobreajuste** (gap 1.8% frente a ~60% del RF), pero **no fue seleccionada como modelo operativo** debido a que presentó un desempeño predictivo inferior en test.

- **Matiz estadístico honesto:** la diferencia de desempeño entre los modelos **no es estadísticamente significativa** (prueba *t* pareada, $p \approx 0.13$); las diferencias de RMSE en test entre las mejores configuraciones están dentro del ruido de muestreo. La elección del Random Forest se sustenta en su mejor desempeño puntual en test y en la robustez del *ensemble*; la transformación log1p es reproducible con `python -m src.train --log-target` y se conserva documentada como alternativa.
- Ambos modelos se muestran al usuario para una decisión informada.
- **Mejoras futuras:** incorporar más variables del dataset, usar `permutation_importance`, y validar el modelo con datos de otras regiones/períodos; evaluar log1p como línea de mejora de calibración del modelo lineal.

9. Referencias

- Dataset: <https://www.kaggle.com/competitions/house-prices-advanced-regression-techniques/data>
- Métricas de regresión (MAE, MSE, RMSE): <https://www.datacamp.com/es/tutorial/rmse>